MANUAL DE IMPLEMENTAÇÃO

SINC — Sistema Inteligente de Notificação e Captura

Documentação Técnica para Desenvolvimento e Operação

Versão 1.0 — Documentação Técnica
Abril de 2026
Equipe SINC — Diretor de Projetos: Bernardo Lopes

Documento de uso interno da equipe de desenvolvimento.

1. Introdução Técnica

1.1 Sobre este documento

Este documento é o manual técnico de implementação do SINC. Cobre arquitetura, componentes, código, configuração, deploy e operação. Não cobre aspectos contratuais, jurídicos ou comerciais — para esses, consultar o Documento de Arquitetura Técnica completo.

Audiência: equipe de desenvolvimento, equipe de manutenção, novos integrantes que precisem de onboarding técnico.

1.2 O que o sistema faz, em uma frase

Recebe streams RTSP de câmeras, processa cada frame com modelos de IA (detecção de pessoa, reconhecimento facial contra watchlist, análise comportamental, Re-ID multi-câmera), gera alertas para operadores humanos, e persiste apenas o que é confirmado.

1.3 Princípios técnicos que guiam o código

- **Humano no laço:** nenhum endpoint do sistema dispara ação física automaticamente. Todo alerta espera confirmação.
- **Minimização:** default é descartar; persistir é exceção. Embeddings de não-match vivem <1s em RAM.
- **Degradação graciosa:** falha de componente não derruba o sistema; falha de tracking é reportada honestamente, não escondida.
- **Auditabilidade:** toda decisão (humana ou de sistema) gera log estruturado e/ou entrada em audit_log.
- **Modularidade:** cada componente (detector, embedder, tracker) é substituível sem reescrita. Comunicação via filas e contratos bem-definidos.

1.4 Camadas funcionais

- **Camada 1 — Filtro Facial:** ArcFace contra watchlist (~50 pessoas) em câmeras de entrada/saída.
- **Camada 2 — Análise Comportamental:** RTMPose + regras explícitas em câmeras de salão.
- **Camada 3 — Tracking Dirigido:** OSNet ativado APENAS para indivíduos já flaggados pelas Camadas 1 ou 2.

- **Camada 4 — Validação Humana:** dashboard + notificação móvel; operador decide.
- **Camada 5 — Persistência:** PostgreSQL apenas para eventos confirmados.

2. Arquitetura e Estrutura do Projeto

2.1 Visão geral em diagrama

```
Câmera Intelbras (RTSP)
    │
    ▼
[ingest] ── decodifica ──▶ fila de frames (Redis Stream)
    │
    ▼
[person_detector] (YOLOv11n) ── pessoa? ──▶ descarta se não
    │
    ▼
[tracker] (ByteTrack) ── Local_ID + bbox
    │
    ├──────────────────▶ CAMADA 1 (entrada/saída)
    │                [face_detector] → [quality_filter] → [face_embedder]
    │                                                          │
    │                                                          ▼
    │                                                  [faiss_search] ──▶ alerta?
    │
    └──────────────────▶ CAMADA 2 (salão)
                     [pose_estimator] → [behavior_analyzer] ──▶ alerta?

                            ▼ (alerta gerado)
                     [body_embedder] (OSNet)
                            │
                            ▼
                     [tracking_service] (Redis pub/sub)
                            │
                            ▼
            [api] ──▶ WebSocket ──▶ [dashboard]
                  ──▶ FCM/Telegram ──▶ [app segurança]

                            ▼ (decisão humana)
                     [postgresql] (eventos confirmados + auditoria)
```

2.2 Estrutura de diretórios do repositório

```
sinc/
├── README.md
├── docker-compose.yml
├── docker-compose.prod.yml
├── .env.example
```

```
├── pyproject.toml                    # gerenciado via Poetry ou uv
├── requirements.txt                  # alternativa simples
│
├── docs/                             # documentação técnica
│   ├── implementacao.docx            # este documento
│   ├── arquitetura.docx              # documento completo
│   └── runbooks/                     # procedimentos operacionais
│
├── infra/
│   ├── postgres/
│   │   └── init.sql                  # schema inicial
│   ├── prometheus/
│   │   └── prometheus.yml
│   ├── grafana/
│   │   └── dashboards/
│   └── loki/
│       └── config.yml
│
├── models/                           # baixados em setup, não versionados
│   ├── yolov11n.onnx
│   ├── yunet_2023mar.onnx
│   ├── arcface_buffalo_l/
│   ├── osnet_x1_0.pth
│   └── rtmpose_m.pth
│
├── sinc/                             # pacote Python principal
│   ├── __init__.py
│   ├── config.py                     # configuração via Pydantic Settings
│   ├── logger.py                     # configuração structlog
│   │
│   ├── ingest/
│   │   ├── __init__.py
│   │   ├── rtsp_reader.py            # captura RTSP por câmera
│   │   └── frame_queue.py            # fila Redis Stream
│   │
│   ├── pipeline/
│   │   ├── __init__.py
│   │   ├── orchestrator.py           # coordena pipeline por câmera
│   │   ├── person_detector.py        # YOLOv11n
│   │   ├── tracker.py                # ByteTrack wrapper
│   │   ├── face_detector.py          # YuNet
│   │   ├── face_quality.py           # filtro de qualidade
│   │   ├── face_embedder.py          # ArcFace via InsightFace
│   │   ├── body_embedder.py          # OSNet via Torchreid
│   │   ├── pose_estimator.py         # RTMPose
│   │   └── behavior/
│   │       ├── __init__.py
│   │       ├── linger_detector.py    # Detector A
│   │       ├── hide_pose_detector.py # Detector B
```

```
│   │       ├── package_detector.py   # Detector C
│   │       └── motion_detector.py    # Detector D
│   │
│   ├── matching/
│   │   ├── __init__.py
│   │   ├── faiss_index.py            # galeria de procurados
│   │   └── tracking_service.py       # Re-ID ativo (Redis pub/sub)
│   │
│   ├── alerts/
│   │   ├── __init__.py
│   │   ├── manager.py                # criação e ciclo de vida
│   │   └── notifier.py               # FCM, Telegram, email
│   │
│   ├── api/
│   │   ├── __init__.py
│   │   ├── main.py                   # FastAPI app
│   │   ├── auth.py                   # JWT, papéis
│   │   ├── routes/
│   │   │   ├── alerts.py
│   │   │   ├── watchlist.py
│   │   │   ├── cameras.py
│   │   │   ├── users.py
│   │   │   └── health.py
│   │   ├── websocket.py
│   │   └── schemas/                  # Pydantic models
│   │
│   ├── db/
│   │   ├── __init__.py
│   │   ├── models.py                 # SQLAlchemy
│   │   ├── session.py
│   │   └── migrations/               # Alembic
│   │
│   ├── workers/
│   │   ├── __init__.py
│   │   ├── retention_job.py          # purga de dados expirados
│   │   ├── backup_job.py
│   │   └── metrics_job.py
│   │
│   └── utils/
│       ├── __init__.py
│       └── audit.py                  # helpers de audit_log
│
├── frontend/                         # React + TypeScript + Vite
│   ├── package.json
│   ├── vite.config.ts
│   ├── tsconfig.json
│   ├── public/
│   └── src/
│       ├── main.tsx
```

```
│        ├── App.tsx
│        ├── api/                      # cliente HTTP + WS
│        ├── components/
│        │   ├── AlertList.tsx
│        │   ├── AlertDetail.tsx
│        │   ├── TrackingMap.tsx
│        │   └── ui/                   # shadcn/ui
│        ├── pages/
│        │   ├── Dashboard.tsx
│        │   ├── Watchlist.tsx
│        │   ├── Audit.tsx
│        │   └── Login.tsx
│        ├── hooks/
│        └── stores/                   # Zustand
│
├── mobile/                            # React Native (app do segurança)
│   └── ...
│
├── scripts/
│   ├── download_models.sh             # baixa modelos pré-treinados
│   ├── seed_dev_data.py               # dados de desenvolvimento
│   ├── benchmark.py                   # smoke test de inferência
│   └── calibrate_threshold.py         # calibração de thresholds
│
└── tests/
    ├── unit/
    ├── integration/
    └── fixtures/
        └── videos/                    # vídeos de bancada para testes
```

2.3 Comunicação entre componentes

- **Redis Streams** para filas entre estágios do pipeline (frames brutos → frames com pessoas → frames com faces, etc.).
- **Redis Pub/Sub** para tracking ativo: quando uma câmera flagga uma pessoa, publica o body_embedding em canal que outras câmeras assinam.
- **PostgreSQL** para tudo que precisa de persistência durável (alerts, watchlist, audit_log).
- **WebSocket (FastAPI)** para o dashboard receber alertas e atualizações de tracking em tempo real.
- **HTTP REST (FastAPI)** para operações administrativas (CRUD watchlist, listagem histórica, login, etc.).

3. Stack e Dependências

3.1 Tecnologias principais

Camada	Tecnologia	Versão alvo
Linguagem backend	Python	3.11.x
Web framework	FastAPI	>=0.110, <1.0
ASGI server	Uvicorn	>=0.27
Banco relacional	PostgreSQL	16.x
ORM + migrations	SQLAlchemy + Alembic	2.0+ / 1.13+
Cache + filas	Redis	7.x
Detecção de pessoa	Ultralytics YOLOv11n	ultralytics>=8.2
Tracking	ByteTrack (via Ultralytics)	integrado
Detecção facial	YuNet (OpenCV)	opencv-python>=4.9
Embedding facial	InsightFace buffalo_l	insightface>=0.7
Embedding corpo	Torchreid OSNet x1_0	torchreid>=1.4
Pose estimation	MMPose RTMPose-m	mmpose>=1.3
Busca vetorial	FAISS CPU	faiss-cpu>=1.8
Frontend	React + TS + Vite + Tailwind + shadcn/ui	React 18, TS 5
Métricas / logs	Prometheus + Grafana + Loki	Prom 2.50+
Containerização	Docker + Docker Compose	Docker 26+
Sistema operacional	Ubuntu Server LTS	24.04

3.2 requirements.txt (referência)

```
# === Web framework ===
fastapi>=0.110,<1.0
uvicorn[standard]>=0.27
pydantic>=2.6
```

```
pydantic-settings>=2.2
python-multipart>=0.0.9

# === Auth ===
python-jose[cryptography]>=3.3
passlib[argon2]>=1.7.4

# === Banco ===
sqlalchemy>=2.0
alembic>=1.13
psycopg[binary]>=3.1
redis>=5.0

# === Visão computacional ===
opencv-python>=4.9
numpy>=1.26
torch>=2.2
torchvision>=0.17
ultralytics>=8.2
insightface>=0.7
onnxruntime-gpu>=1.17  # ou onnxruntime para CPU
faiss-cpu>=1.8
torchreid>=1.4
mmpose>=1.3
mmcv>=2.1

# === Observabilidade ===
structlog>=24.1
prometheus-client>=0.20

# === Notificações ===
firebase-admin>=6.4
httpx>=0.27  # Telegram via API HTTP

# === Dev ===
pytest>=8.0
pytest-asyncio>=0.23
black>=24.0
ruff>=0.3
mypy>=1.9
```

3.3 Modelos pré-treinados

Modelos não são versionados no repositório (são grandes e binários). Existe um script que baixa todos eles na primeira execução:

```
# scripts/download_models.sh
#!/bin/bash
set -euo pipefail
```

```
MODELS_DIR="${MODELS_DIR:-./models}"
mkdir -p "$MODELS_DIR"

echo "==> Baixando YOLOv11n..."
python -c "from ultralytics import YOLO; YOLO('yolo11n.pt')"
mv yolo11n.pt "$MODELS_DIR/"

echo "==> Baixando YuNet..."
wget -nc -O "$MODELS_DIR/face_detection_yunet_2023mar.onnx" \

https://github.com/opencv/opencv_zoo/raw/main/models/face_detection_yunet/face_detection_y
unet_2023mar.onnx

echo "==> Baixando InsightFace buffalo_l..."
python -c "import insightface; app = insightface.app.FaceAnalysis(name='buffalo_l');
app.prepare(ctx_id=0)"

echo "==> Baixando OSNet..."
python -c "from torchreid.utils import load_pretrained_weights; \
  from torchreid.models import build_model; \
  m = build_model('osnet_x1_0', 1000, pretrained=True)"

echo "==> Concluído."
```

4. Componentes Principais

Esta seção mostra a forma esperada de cada componente do pipeline com exemplos de código de referência. O código nos exemplos é simplificado: na implementação real terá tratamento de erros, logging, métricas e configuração externa.

4.1 Captura RTSP

Cada câmera roda em sua própria task asyncio que mantém a conexão RTSP, faz watchdog de timeout e empilha frames decodificados em uma fila.

```
# sinc/ingest/rtsp_reader.py
import cv2
import asyncio
from sinc.logger import logger

class RTSPReader:
    def __init__(self, camera_id: str, rtsp_url: str, target_fps: int = 5):
        self.camera_id = camera_id
        self.rtsp_url = rtsp_url
        self.target_fps = target_fps
        self.cap = None
```

```
        self.running = False

    async def start(self, on_frame):
        self.running = True
        while self.running:
            try:
                self.cap = cv2.VideoCapture(self.rtsp_url, cv2.CAP_FFMPEG)
                if not self.cap.isOpened():
                    logger.warning("rtsp.open_failed", camera=self.camera_id)
                    await asyncio.sleep(5)
                    continue

                native_fps = self.cap.get(cv2.CAP_PROP_FPS) or 30
                skip = max(1, int(native_fps // self.target_fps))
                frame_count = 0

                while self.running:
                    ret, frame = self.cap.read()
                    if not ret:
                        logger.warning("rtsp.read_failed", camera=self.camera_id)
                        break
                    frame_count += 1
                    if frame_count % skip == 0:
                        await on_frame(self.camera_id, frame)

            except Exception as e:
                logger.error("rtsp.exception", camera=self.camera_id, error=str(e))
            finally:
                if self.cap:
                    self.cap.release()
                if self.running:
                    await asyncio.sleep(2)  # backoff antes de reconectar

    def stop(self):
        self.running = False
```

4.2 Detecção de pessoa + tracker

```
# sinc/pipeline/person_detector.py
from ultralytics import YOLO

class PersonDetector:
    def __init__(self, model_path: str, conf: float = 0.4):
        self.model = YOLO(model_path)
        self.conf = conf

    def detect(self, frame, camera_id: str):
        # tracker integrado mantém Local_ID estável entre frames
        results = self.model.track(
```

```
            frame, persist=True, classes=[0],  # 0 = person
            conf=self.conf, tracker="bytetrack.yaml", verbose=False
        )
        detections = []
        if results[0].boxes is not None:
            boxes = results[0].boxes
            for i in range(len(boxes)):
                if boxes.id is None:
                    continue
                xyxy = boxes.xyxy[i].cpu().numpy()
                track_id = int(boxes.id[i].item())
                conf_score = float(boxes.conf[i].item())
                detections.append({
                    "camera_id": camera_id,
                    "local_id": track_id,
                    "bbox": tuple(xyxy.astype(int).tolist()),
                    "conf": conf_score,
                })
        return detections
```

4.3 Filtro de qualidade facial

```
# sinc/pipeline/face_quality.py
import cv2
import numpy as np

class FaceQualityFilter:
    def __init__(self, min_size=80, min_sharpness=60.0,
                 max_yaw_deg=30, max_pitch_deg=25):
        self.min_size = min_size
        self.min_sharpness = min_sharpness
        self.max_yaw = max_yaw_deg
        self.max_pitch = max_pitch_deg

    def passes(self, face_crop, keypoints) -> tuple[bool, dict]:
        h, w = face_crop.shape[:2]
        # 1. tamanho
        if min(h, w) < self.min_size:
            return False, {"reason": "size", "size": min(h, w)}

        # 2. nitidez via Laplaciano
        gray = cv2.cvtColor(face_crop, cv2.COLOR_BGR2GRAY)
        sharpness = cv2.Laplacian(gray, cv2.CV_64F).var()
        if sharpness < self.min_sharpness:
            return False, {"reason": "blur", "sharpness": sharpness}

        # 3. iluminação
        mean_lum = float(gray.mean())
        if mean_lum < 50 or mean_lum > 220:
```

```
            return False, {"reason": "lighting", "mean": mean_lum}

        # 4. pose (estimativa simples via keypoints)
        yaw, pitch = self._estimate_pose(keypoints, w, h)
        if abs(yaw) > self.max_yaw or abs(pitch) > self.max_pitch:
            return False, {"reason": "pose", "yaw": yaw, "pitch": pitch}

        return True, {"sharpness": sharpness, "mean_lum": mean_lum,
                      "yaw": yaw, "pitch": pitch}

    def _estimate_pose(self, kpts, w, h):
        # kpts = [(le_x, le_y), (re_x, re_y), (nose_x, nose_y), ...]
        le, re, nose = kpts[0], kpts[1], kpts[2]
        eyes_mid_x = (le[0] + re[0]) / 2
        yaw = (nose[0] - eyes_mid_x) / (w / 2) * 45  # aproximação grosseira
        eyes_mid_y = (le[1] + re[1]) / 2
        pitch = (nose[1] - eyes_mid_y - h * 0.15) / (h / 2) * 45
        return yaw, pitch
```

4.4 Embedding e busca FAISS

```
# sinc/matching/faiss_index.py
import faiss
import numpy as np

class WatchlistIndex:
    def __init__(self, dim: int = 512):
        self.dim = dim
        self.index = faiss.IndexFlatIP(dim)
        self.embedding_to_person = []  # paralelo ao índice

    def add_person_embeddings(self, person_id: str, embeddings: np.ndarray):
        # embeddings: shape (N, 512), já L2-normalizados
        self.index.add(embeddings.astype(np.float32))
        for _ in range(len(embeddings)):
            self.embedding_to_person.append(person_id)

    def search(self, query: np.ndarray, threshold: float = 0.55,
               margin: float = 0.05) -> dict | None:
        # query: shape (1, 512), L2-normalizado
        if self.index.ntotal == 0:
            return None
        scores, idxs = self.index.search(query.astype(np.float32),
                                         k=min(5, self.index.ntotal))
        scores, idxs = scores[0], idxs[0]

        # melhor por pessoa
        best_per_person = {}
        for s, i in zip(scores, idxs):
```

```
            pid = self.embedding_to_person[i]
            if pid not in best_per_person or s > best_per_person[pid]:
                best_per_person[pid] = float(s)

        ranked = sorted(best_per_person.items(), key=lambda x: -x[1])
        if not ranked or ranked[0][1] < threshold:
            return None
        if len(ranked) >= 2 and (ranked[0][1] - ranked[1][1]) < margin:
            return None  # ratio test falhou, pouco discriminativo

        return {"person_id": ranked[0][0], "score": ranked[0][1]}
```

4.5 Detector comportamental — exemplo (permanência anômala)

```
# sinc/pipeline/behavior/linger_detector.py
from collections import deque
from time import time

class LingerDetector:
    """Detecta pessoa parada em uma região por tempo acima do esperado."""

    def __init__(self, window_seconds=60, max_displacement_ratio=0.05,
                 min_linger_seconds=45):
        self.window = window_seconds
        self.max_disp = max_displacement_ratio  # fração da diagonal do frame
        self.min_linger = min_linger_seconds
        self.history = {}  # local_id -> deque[(timestamp, cx, cy)]
        self.alerted = set()

    def update(self, camera_id, local_id, bbox, frame_shape):
        h, w = frame_shape[:2]
        diag = (h**2 + w**2) ** 0.5
        x1, y1, x2, y2 = bbox
        cx, cy = (x1 + x2) / 2, (y1 + y2) / 2
        now = time()

        key = (camera_id, local_id)
        if key not in self.history:
            self.history[key] = deque()
        self.history[key].append((now, cx, cy))

        # remove entradas antigas
        while self.history[key] and now - self.history[key][0][0] > self.window:
            self.history[key].popleft()

        # avalia
        if key in self.alerted:
            return None
        oldest_t, ox, oy = self.history[key][0]
```

```
        elapsed = now - oldest_t
        if elapsed < self.min_linger:
            return None
        disp = ((cx - ox)**2 + (cy - oy)**2) ** 0.5 / diag
        if disp <= self.max_disp:
            self.alerted.add(key)
            return {
                "type": "BEHAVIOR_LINGER",
                "camera_id": camera_id,
                "local_id": local_id,
                "elapsed_seconds": elapsed,
                "displacement_ratio": disp,
                "score": min(1.0, elapsed / 120),  # heurística simples
            }
        return None
```

4.6 Tracking ativo (Re-ID dirigido)

Quando uma pessoa é flaggada, o tracking_service mantém os body_embeddings dela em Redis e cada câmera assina o canal para tentar re-âncora quando alguém aparece com aparência similar.

```
# sinc/matching/tracking_service.py
import json
import numpy as np
import redis.asyncio as aioredis

CHANNEL = "sinc:active_tracking"
KEY_PREFIX = "sinc:tracking:"

class TrackingService:
    def __init__(self, redis: aioredis.Redis):
        self.redis = redis

    async def start_tracking(self, alert_id: str, person_id: str | None,
                             body_embedding: np.ndarray):
        data = {
            "alert_id": alert_id,
            "person_id": person_id,
            "embeddings": [body_embedding.tolist()],
            "started_at": _now_iso(),
            "last_seen": _now_iso(),
            "last_camera": None,
            "confidence": "HIGH",
        }
        await self.redis.set(f"{KEY_PREFIX}{alert_id}",
                             json.dumps(data), ex=900)  # 15min TTL
        await self.redis.publish(CHANNEL, json.dumps({
            "event": "TRACKING_STARTED", "alert_id": alert_id,
```

```
        }))

    async def try_reanchor(self, camera_id: str, candidate_emb: np.ndarray):
        """Cada câmera chama isto para cada pessoa nova detectada
        (mas SOMENTE comparando contra alertas ativos, não watchlist toda)."""
        async for key in self.redis.scan_iter(f"{KEY_PREFIX}*"):
            raw = await self.redis.get(key)
            if not raw:
                continue
            data = json.loads(raw)
            embs = np.array(data["embeddings"])
            sims = embs @ candidate_emb / (
                np.linalg.norm(embs, axis=1) * np.linalg.norm(candidate_emb))
            if sims.max() > 0.85:
                data["last_seen"] = _now_iso()
                data["last_camera"] = camera_id
                data["confidence"] = "HIGH"
                # acumula até 20 embeddings
                if len(data["embeddings"]) < 20:
                    data["embeddings"].append(candidate_emb.tolist())
                await self.redis.set(key, json.dumps(data), ex=900)
                return data["alert_id"]
        return None
```

4.7 Estados de alerta — implementação

```
# sinc/alerts/manager.py
from enum import Enum

class AlertStatus(str, Enum):
    PENDING = "PENDING"
    UNDER_REVIEW = "UNDER_REVIEW"
    CONFIRMED_TRUE = "CONFIRMED_TRUE"
    CONFIRMED_FALSE_POSITIVE = "CONFIRMED_FALSE_POSITIVE"
    CLOSED_INDETERMINATE = "CLOSED_INDETERMINATE"
    EXPIRED = "EXPIRED"

ALLOWED_TRANSITIONS = {
    AlertStatus.PENDING: {AlertStatus.UNDER_REVIEW, AlertStatus.EXPIRED},
    AlertStatus.UNDER_REVIEW: {
        AlertStatus.PENDING,
        AlertStatus.CONFIRMED_TRUE,
        AlertStatus.CONFIRMED_FALSE_POSITIVE,
        AlertStatus.CLOSED_INDETERMINATE,
    },
    # estados terminais não transitam (exceto reabertura admin, fora deste fluxo)
    AlertStatus.CONFIRMED_TRUE: set(),
    AlertStatus.CONFIRMED_FALSE_POSITIVE: set(),
    AlertStatus.CLOSED_INDETERMINATE: set(),
```

```
    AlertStatus.EXPIRED: set(),
}

class InvalidTransition(Exception):
    pass

def transition(current: AlertStatus, target: AlertStatus):
    if target not in ALLOWED_TRANSITIONS[current]:
        raise InvalidTransition(f"{current} -> {target} não permitido")
```

5. Banco de Dados

5.1 Esquema essencial

As tabelas abaixo cobrem o necessário para o MVP. Tabelas adicionais (relatórios, métricas históricas, configuração) são adicionadas conforme o sistema evolui.

5.1.1 Watchlist (persons + photos + embeddings)

```
CREATE TABLE persons (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    internal_code   VARCHAR(32) UNIQUE NOT NULL,
    display_name    VARCHAR(120),
    notes           TEXT,
    status          VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
    inclusion_reason TEXT NOT NULL,
    included_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    included_by     UUID NOT NULL REFERENCES users(id),
    review_due_at   TIMESTAMPTZ,
    excluded_at     TIMESTAMPTZ,
    excluded_by     UUID REFERENCES users(id),
    exclusion_reason TEXT
);
CREATE INDEX idx_persons_status ON persons(status);

CREATE TABLE person_photos (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    person_id       UUID NOT NULL REFERENCES persons(id) ON DELETE CASCADE,
    image_path      TEXT NOT NULL,
    image_hash      VARCHAR(64) UNIQUE NOT NULL,
    quality_score   FLOAT,
    captured_at     TIMESTAMPTZ,
    source          VARCHAR(60),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    created_by      UUID NOT NULL REFERENCES users(id)
);

CREATE TABLE person_embeddings (
```

```
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    person_id       UUID NOT NULL REFERENCES persons(id) ON DELETE CASCADE,
    photo_id        UUID NOT NULL REFERENCES person_photos(id) ON DELETE CASCADE,
    model_name      VARCHAR(40) NOT NULL,
    model_version   VARCHAR(20) NOT NULL,
    embedding       BYTEA NOT NULL,    -- 512 floats = 2048 bytes
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_embeddings_person ON person_embeddings(person_id);
```

5.1.2 Alertas e evidências

```
CREATE TABLE alerts (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    type            VARCHAR(40) NOT NULL,
    status          VARCHAR(20) NOT NULL DEFAULT 'PENDING',
    severity        VARCHAR(10) NOT NULL DEFAULT 'MEDIUM',
    score           FLOAT,
    camera_id       UUID NOT NULL REFERENCES cameras(id),
    person_id       UUID REFERENCES persons(id),
    candidate_score FLOAT,
    behavior_details JSONB,
    detected_at     TIMESTAMPTZ NOT NULL,
    notified_at     TIMESTAMPTZ,
    opened_at       TIMESTAMPTZ,
    closed_at       TIMESTAMPTZ,
    handled_by      UUID REFERENCES users(id),
    closure_note    TEXT
);
CREATE INDEX idx_alerts_status_detected ON alerts(status, detected_at DESC);
CREATE INDEX idx_alerts_camera ON alerts(camera_id, detected_at DESC);

CREATE TABLE alert_evidence (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    alert_id        UUID NOT NULL REFERENCES alerts(id) ON DELETE CASCADE,
    kind            VARCHAR(20) NOT NULL,
    storage_path    TEXT,
    metadata        JSONB,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    retain_until    TIMESTAMPTZ NOT NULL
);
CREATE INDEX idx_evidence_purge ON alert_evidence(retain_until);
```

5.1.3 Usuários, câmeras e auditoria

```
CREATE TABLE users (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    username        VARCHAR(60) UNIQUE NOT NULL,
    full_name       VARCHAR(120) NOT NULL,
    email           VARCHAR(120) UNIQUE,
```

```
    role            VARCHAR(20) NOT NULL,  -- OPERATOR|SECURITY|MANAGER|ADMIN
    password_hash   VARCHAR(120) NOT NULL,
    is_active       BOOLEAN NOT NULL DEFAULT TRUE,
    last_login_at   TIMESTAMPTZ,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE cameras (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    code            VARCHAR(20) UNIQUE NOT NULL,
    description     VARCHAR(120),
    area            VARCHAR(40),
    rtsp_url        TEXT NOT NULL,
    layer1_enabled  BOOLEAN NOT NULL DEFAULT FALSE,
    layer2_enabled  BOOLEAN NOT NULL DEFAULT FALSE,
    layer3_enabled  BOOLEAN NOT NULL DEFAULT TRUE,
    is_active       BOOLEAN NOT NULL DEFAULT TRUE,
    config          JSONB
);

CREATE TABLE audit_log (
    id              BIGSERIAL PRIMARY KEY,
    occurred_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    actor_id        UUID REFERENCES users(id),
    actor_role      VARCHAR(20),
    action          VARCHAR(60) NOT NULL,
    target_kind     VARCHAR(40),
    target_id       UUID,
    details         JSONB,
    ip_address      INET,
    user_agent      TEXT
);
CREATE INDEX idx_audit_occurred ON audit_log(occurred_at DESC);
-- Apenas INSERT autorizado para qualquer role;
-- DELETE/UPDATE bloqueados via GRANT.
```

5.2 Migrations

Usar Alembic. Cada mudança de schema gera um arquivo de migration versionado em sinc/db/migrations/. Ordem básica de comandos:

```
# criar nova migration a partir de mudanças nos models SQLAlchemy
alembic revision --autogenerate -m "add behavior_details to alerts"

# aplicar todas migrations pendentes
alembic upgrade head

# rollback uma versão
alembic downgrade -1
```

5.3 Job de retenção

```
# sinc/workers/retention_job.py
from sqlalchemy import delete, select
from sinc.db.models import AlertEvidence
from sinc.logger import logger
import os, asyncio
from datetime import datetime

async def run_retention(session):
    """Executar diariamente (cron interno)."""
    expired = await session.execute(
        select(AlertEvidence).where(AlertEvidence.retain_until < datetime.utcnow())
    )
    rows = expired.scalars().all()
    deleted = 0
    for ev in rows:
        try:
            if ev.storage_path and os.path.exists(ev.storage_path):
                os.remove(ev.storage_path)
            await session.delete(ev)
            deleted += 1
        except Exception as e:
            logger.error("retention.failed", evidence_id=str(ev.id), err=str(e))
    await session.commit()
    logger.info("retention.summary", deleted=deleted, scanned=len(rows))
```

6. API REST e WebSocket

6.1 Endpoints principais

Método	Caminho	Papel mín.	Função
POST	`/api/auth/login`	—	Autentica usuário, retorna JWT
POST	`/api/auth/refresh`	—	Renova access token via refresh
GET	`/api/alerts?status=PENDING&limit=50`	OPERATOR	Lista alertas, filtros e paginação
GET	`/api/alerts/{id}`	OPERATOR	Detalhe completo do alerta
PATCH	`/api/alerts/{id}/status`	OPERATOR	Transição de estado (com validação)
GET	`/api/alerts/{id}/video`	OPERATOR	Stream HLS do vídeo associado
GET	`/api/watchlist`	MANAGER	Lista pessoas na watchlist
POST	`/api/watchlist`	MANAGER	Inclui pessoa (multipart com fotos)

Método	Caminho	Papel mín.	Função
DELETE	`/api/watchlist/{id}`	MANAGER	Exclui pessoa (com justificativa)
GET	`/api/cameras`	OPERATOR	Lista câmeras e status
GET	`/api/health`	—	Healthcheck (k8s/docker)
WS	`/ws/operator`	OPERATOR	Canal em tempo real para alertas/tracking

6.2 Exemplo de schema (Pydantic)

```
# sinc/api/schemas/alert.py
from pydantic import BaseModel, Field
from datetime import datetime
from uuid import UUID

class AlertOut(BaseModel):
    id: UUID
    type: str
    status: str
    severity: str
    score: float | None
    camera_id: UUID
    camera_code: str
    person_id: UUID | None
    person_code: str | None
    detected_at: datetime
    opened_at: datetime | None
    closed_at: datetime | None
    behavior_details: dict | None

class AlertStatusUpdate(BaseModel):
    target_status: str = Field(...,

pattern=r"^(UNDER_REVIEW|CONFIRMED_TRUE|CONFIRMED_FALSE_POSITIVE|CLOSED_INDETERMINATE|PEND
ING)$")
    closure_note: str | None = Field(None, max_length=500)
```

6.3 Exemplo de rota

```
# sinc/api/routes/alerts.py
from fastapi import APIRouter, Depends, HTTPException
from sinc.api.auth import require_role, current_user
from sinc.api.schemas.alert import AlertOut, AlertStatusUpdate
from sinc.alerts.manager import transition, AlertStatus
from sinc.utils.audit import write_audit
```

```
router = APIRouter(prefix="/api/alerts", tags=["alerts"])

@router.patch("/{alert_id}/status", response_model=AlertOut)
async def update_status(
    alert_id: UUID,
    payload: AlertStatusUpdate,
    user=Depends(require_role("OPERATOR")),
    session=Depends(get_session),
):
    alert = await session.get(Alert, alert_id)
    if not alert:
        raise HTTPException(404, "alerta não encontrado")
    try:
        transition(AlertStatus(alert.status), AlertStatus(payload.target_status))
    except InvalidTransition as e:
        raise HTTPException(400, str(e))

    alert.status = payload.target_status
    alert.handled_by = user.id
    alert.closure_note = payload.closure_note
    if payload.target_status in ("CONFIRMED_TRUE", "CONFIRMED_FALSE_POSITIVE",
                                 "CLOSED_INDETERMINATE"):
        alert.closed_at = datetime.utcnow()
    await session.commit()

    await write_audit(session, user, "ALERT_STATUS_CHANGE",
                      target_kind="ALERT", target_id=alert.id,
                      details={"to": payload.target_status})
    return AlertOut.model_validate(alert)
```

6.4 WebSocket — fluxo simplificado

```
# sinc/api/websocket.py (esqueleto)
@app.websocket("/ws/operator")
async def operator_ws(ws: WebSocket, user = Depends(ws_authenticate)):
    await ws.accept()
    pubsub = redis.pubsub()
    await pubsub.subscribe("sinc:alerts:new", "sinc:tracking:update")
    try:
        async for message in pubsub.listen():
            if message["type"] != "message":
                continue
            await ws.send_json(json.loads(message["data"]))
    except WebSocketDisconnect:
        pass
    finally:
        await pubsub.unsubscribe()
```

7. Deploy e Execução

7.1 Pré-requisitos do servidor

- Ubuntu Server 24.04 LTS (ou 22.04)
- Drivers NVIDIA + CUDA toolkit (se com GPU)
- Docker Engine 26+ e Docker Compose v2
- Pelo menos 32 GB RAM e SSD de 500 GB para MVP; ver doc completo de arquitetura para Fase 4
- Acesso de rede às câmeras (VLAN dedicada recomendada)

7.2 docker-compose.yml (referência)

```
# docker-compose.yml
services:

  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: sinc
      POSTGRES_USER: sinc
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./infra/postgres/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U sinc"]
      interval: 5s
    restart: unless-stopped

  redis:
    image: redis:7-alpine
    command: ["redis-server", "--appendonly", "yes"]
    volumes:
      - redisdata:/data
    restart: unless-stopped

  api:
    build:
      context: .
      dockerfile: Dockerfile.api
    env_file: .env
    depends_on:
      postgres:
        condition: service_healthy
```

```
      redis:
        condition: service_started
    ports:
      - "8000:8000"
    restart: unless-stopped

  pipeline:
    build:
      context: .
      dockerfile: Dockerfile.pipeline
    env_file: .env
    depends_on: [postgres, redis]
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: all
              capabilities: [gpu]
    volumes:
      - ./models:/app/models:ro
      - evidence:/app/evidence
    restart: unless-stopped

  frontend:
    build:
      context: ./frontend
    ports:
      - "3000:80"
    restart: unless-stopped

  prometheus:
    image: prom/prometheus:latest
    volumes:
      - ./infra/prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
    restart: unless-stopped

  grafana:
    image: grafana/grafana:latest
    ports:
      - "3001:3000"
    volumes:
      - grafanadata:/var/lib/grafana
    restart: unless-stopped

  loki:
    image: grafana/loki:latest
    volumes:
      - ./infra/loki/config.yml:/etc/loki/config.yml:ro
```

```
    restart: unless-stopped

volumes:
  pgdata:
  redisdata:
  grafanadata:
  evidence:
```

7.3 .env.example

```
# === Banco ===
POSTGRES_PASSWORD=trocar_em_producao
DATABASE_URL=postgresql+psycopg://sinc:trocar_em_producao@postgres:5432/sinc

# === Redis ===
REDIS_URL=redis://redis:6379/0

# === Auth ===
JWT_SECRET=gerar_chave_aleatoria_longa_em_producao
JWT_ACCESS_MINUTES=60
JWT_REFRESH_DAYS=7

# === Modelos ===
MODELS_DIR=/app/models
DEVICE=cuda  # ou cpu

# === Thresholds (calibrar em campo) ===
FACE_MATCH_THRESHOLD=0.55
FACE_MATCH_MARGIN=0.05
FACE_MIN_SIZE=80
FACE_MIN_SHARPNESS=60.0
BODY_REID_THRESHOLD=0.85

# === Retenção (dias) ===
RETENTION_FALSE_POSITIVE_DAYS=30
RETENTION_TRUE_POSITIVE_DAYS=90
RETENTION_AUDIT_LOG_DAYS=365

# === Notificações ===
FCM_PROJECT_ID=...
FCM_CREDENTIALS_PATH=/secrets/fcm.json
TELEGRAM_BOT_TOKEN=...
TELEGRAM_CHAT_ID=...
```

7.4 Fluxo de primeira execução

1. Clonar repositório no servidor.
2. Copiar .env.example para .env e preencher valores reais.

3. Executar scripts/download_models.sh para baixar modelos.
4. Subir banco e Redis: docker compose up -d postgres redis.
5. Aplicar migrations: docker compose run --rm api alembic upgrade head.
6. Criar usuário admin inicial: docker compose run --rm api python -m sinc.cli create-admin.
7. Subir o resto: docker compose up -d.
8. Validar: curl http://localhost:8000/api/health deve retornar OK.
9. Abrir dashboard em http://servidor:3000 e fazer login.
10. Cadastrar primeiras câmeras na interface ou via seed.

7.5 Comandos úteis no dia-a-dia

```
# === Logs ===
docker compose logs -f api
docker compose logs -f pipeline
docker compose logs --since 30m

# === Banco ===
docker compose exec postgres psql -U sinc
\dt        # listar tabelas
SELECT count(*) FROM alerts WHERE detected_at > NOW() - INTERVAL '1 day';

# === Restart de um serviço ===
docker compose restart pipeline

# === Atualizar código e reconstruir ===
git pull
docker compose build api pipeline
docker compose up -d

# === Backup manual do banco ===
docker compose exec postgres pg_dump -U sinc sinc | gzip > backup_$(date +%F).sql.gz
```

8. Testes, Observabilidade e Operação

8.1 Testes

8.1.1 Estrutura

- tests/unit/ — testes de funções puras (filtro de qualidade, máquina de estados, validações).
- tests/integration/ — testes que exercitam banco, Redis, API HTTP. Usam fixtures Docker (testcontainers).

- tests/fixtures/videos/ — vídeos curtos de bancada para testar pipeline ponta-a-ponta sem câmera real.

8.1.2 Exemplos de teste

```
# tests/unit/test_alert_state_machine.py
import pytest
from sinc.alerts.manager import transition, AlertStatus, InvalidTransition

def test_pending_to_under_review_ok():
    transition(AlertStatus.PENDING, AlertStatus.UNDER_REVIEW)

def test_pending_to_confirmed_true_blocked():
    with pytest.raises(InvalidTransition):
        transition(AlertStatus.PENDING, AlertStatus.CONFIRMED_TRUE)

def test_terminal_state_does_not_transition():
    with pytest.raises(InvalidTransition):
        transition(AlertStatus.CONFIRMED_TRUE, AlertStatus.UNDER_REVIEW)
```

```
# tests/integration/test_pipeline_face_match.py
import pytest, cv2
from sinc.pipeline.orchestrator import Pipeline

@pytest.mark.asyncio
async def test_face_match_against_loaded_gallery(pipeline_with_gallery):
    # Gallery contém embedding da pessoa "alice".
    # Vídeo de teste mostra alice entrando no campo de visão.
    cap = cv2.VideoCapture("tests/fixtures/videos/alice_entering.mp4")
    alerts = []
    while cap.isOpened():
        ret, frame = cap.read()
        if not ret: break
        result = await pipeline_with_gallery.process(frame, camera_id="cam-test")
        if result and result.get("type") == "FACE_MATCH":
            alerts.append(result)
    cap.release()
    assert any(a["person_code"] == "alice" for a in alerts)
```

8.1.3 Smoke test de inferência

```
# scripts/benchmark.py
"""Mede FPS de cada estágio do pipeline em hardware atual."""
import time, cv2
from sinc.pipeline.person_detector import PersonDetector
from sinc.pipeline.face_detector import FaceDetector
from sinc.pipeline.face_embedder import FaceEmbedder

frame = cv2.imread("tests/fixtures/sample_frame.jpg")
yolo = PersonDetector("models/yolo11n.pt")
```

```
yunet = FaceDetector("models/face_detection_yunet_2023mar.onnx")
arc = FaceEmbedder()

for name, fn in [
    ("yolo", lambda: yolo.detect(frame, "test")),
    ("yunet", lambda: yunet.detect(frame)),
    ("arcface", lambda: arc.embed(frame)),
]:
    t0 = time.time()
    for _ in range(100): fn()
    dt = (time.time() - t0) / 100
    print(f"{name:10s}: {dt*1000:6.1f} ms/frame -> {1/dt:6.0f} FPS")
```

8.2 Métricas Prometheus

8.2.1 Métricas-chave expostas pela aplicação

```
# sinc/utils/metrics.py
from prometheus_client import Counter, Histogram, Gauge

frames_processed = Counter(
    "sinc_frames_processed_total",
    "Frames processados", ["camera_id", "stage"]
)

stage_latency = Histogram(
    "sinc_stage_latency_seconds",
    "Latência de cada estágio do pipeline",
    ["stage"],
    buckets=(0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1.0, 2.5)
)

alerts_generated = Counter(
    "sinc_alerts_generated_total",
    "Alertas gerados por tipo", ["type"]
)

alerts_open = Gauge(
    "sinc_alerts_open",
    "Alertas com status PENDING ou UNDER_REVIEW"
)

queue_depth = Gauge(
    "sinc_queue_depth",
    "Profundidade da fila Redis Stream", ["stream"]
)

camera_last_frame_age = Gauge(
    "sinc_camera_last_frame_age_seconds",
```

```
    "Idade do último frame recebido", ["camera_id"]
)
```

8.2.2 Painéis Grafana sugeridos

- Saúde por câmera: matriz de câmeras × FPS atual / idade do último frame.
- Latência do pipeline: histograma p50/p95/p99 por estágio.
- Volume de alertas: linha temporal por tipo, com média móvel de 7 dias.
- Taxa de confirmação: % de alertas confirmados verdadeiros vs falso positivos, agregado por câmera.
- Latência operador: tempo entre detected_at e opened_at.
- Recursos do servidor: CPU, RAM, GPU (utilização e VRAM), disco.

8.3 Logs estruturados

```
# sinc/logger.py
import structlog, logging, sys

structlog.configure(
    processors=[
        structlog.contextvars.merge_contextvars,
        structlog.processors.add_log_level,
        structlog.processors.TimeStamper(fmt="iso"),
        structlog.processors.StackInfoRenderer(),
        structlog.processors.format_exc_info,
        structlog.processors.JSONRenderer(),
    ],
    wrapper_class=structlog.make_filtering_bound_logger(logging.INFO),
    logger_factory=structlog.PrintLoggerFactory(file=sys.stdout),
)

logger = structlog.get_logger("sinc")

# uso:
# logger.info("alert.created", alert_id=str(a.id), type=a.type, score=a.score)
```

Logs em JSON são consumidos por Promtail → Loki → Grafana, permitindo busca por correlation_id, camera_id, alert_id em todos os componentes.

8.4 Backup e recuperação

- Banco PostgreSQL: pg_basebackup diário + WAL contínuo, retido por 30 dias.
- Configurações: arquivos em /etc/sinc/ versionados em Git interno.
- Modelos e índice FAISS: backup semanal.

- Evidências: replicação para storage secundário.
- Teste de restore mensal obrigatório — backup não testado não é confiável.

8.5 Atualizações

- Patch (correção de bug, sem mudança de comportamento): janela mensal.
- Minor (novos recursos): janela trimestral.
- Major (breaking changes, modelos novos, schema novo): negociado com o Kan, com plano de rollback documentado.
- Crítico de segurança: aplicado o mais rápido possível após validação.

8.6 Runbooks recomendados

Procedimentos curtos e direcionados, escritos em markdown e versionados em docs/runbooks/. Lista mínima:

- Câmera offline há mais de 5 minutos.
- GPU inacessível / driver caiu.
- PostgreSQL não responde.
- Backup diário falhou.
- Alertas não estão chegando ao dashboard.
- Adicionar nova pessoa à watchlist.
- Adicionar nova câmera ao sistema.
- Calibrar threshold de uma câmera específica.
- Restaurar banco a partir de backup.

8.7 Calibração de thresholds em campo

Após implantar uma câmera, threshold inicial vem do .env. Calibração refinada usa o script abaixo, que opera em modo shadow durante alguns dias e produz curva ROC para escolha do operating point:

```
# scripts/calibrate_threshold.py (uso simplificado)
# 1. Roda em modo shadow por N dias coletando todos os scores
#    e os rótulos atribuídos pelos operadores (TP / FP).
# 2. Plota ROC e PR.
# 3. Sugere threshold no joelho da curva precision-recall,
#    ou no ponto que atinge a precisão alvo (default 0.85).
```

```
python scripts/calibrate_threshold.py \
    --camera-id <UUID> \
    --layer 1 \
    --since "2026-04-01" \
    --target-precision 0.85
```

Fim do documento.